

TERRAFORM – TASK – 05

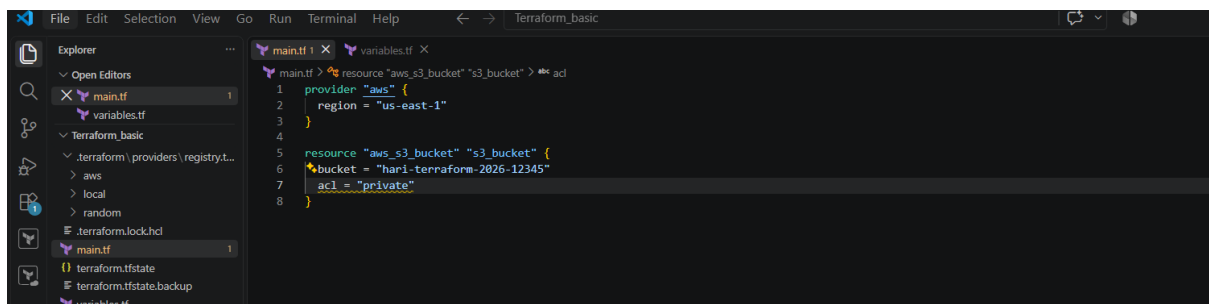
2. Execute the script shown in the video.

- Open VS Code → open your Terraform folder.
- Create/Open main.tf.

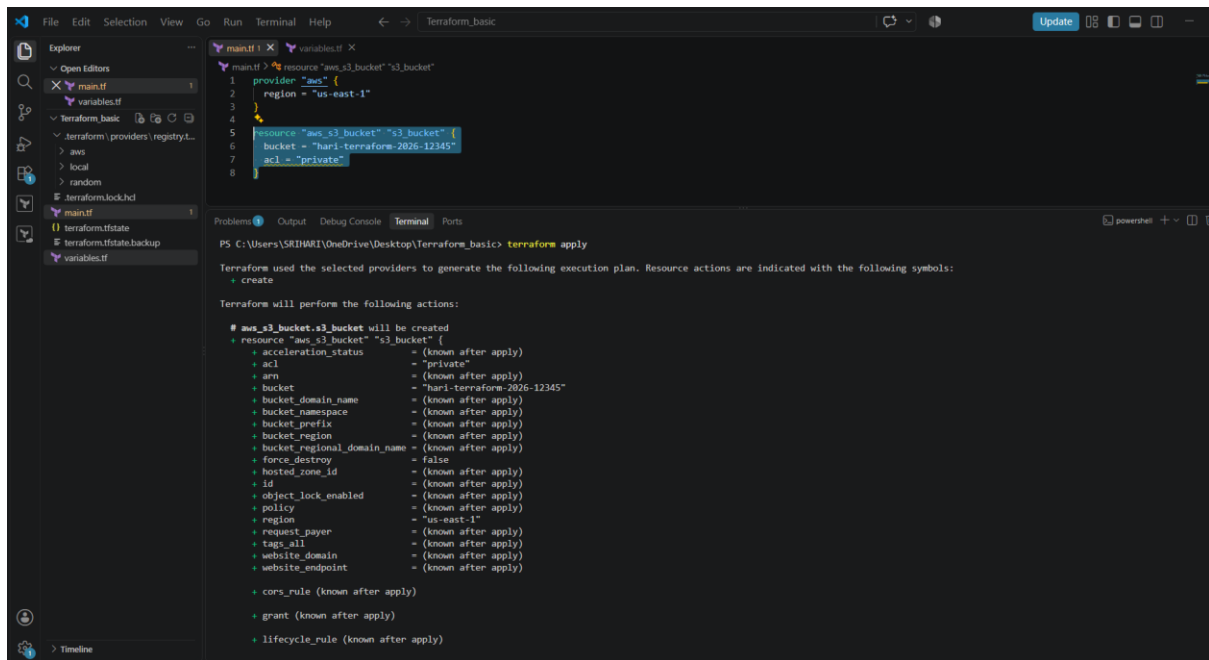
- Add S3 bucket resource code:

```
resource "aws_s3_bucket" "s3_bucket" {  
    bucket = "hari-terraform-2026-12345"  
    acl = "private"  
}
```

- Open Terminal in VS Code.
- Run:
 - terraform init
- Confirm message: **Terraform has been successfully initialized!**



- In VS Code terminal, run:
 - terraform apply
- Terraform shows execution plan (what will be created).
- Check that it shows:
 - aws_s3_bucket.s3_bucket will be created
- Continue to approval step.

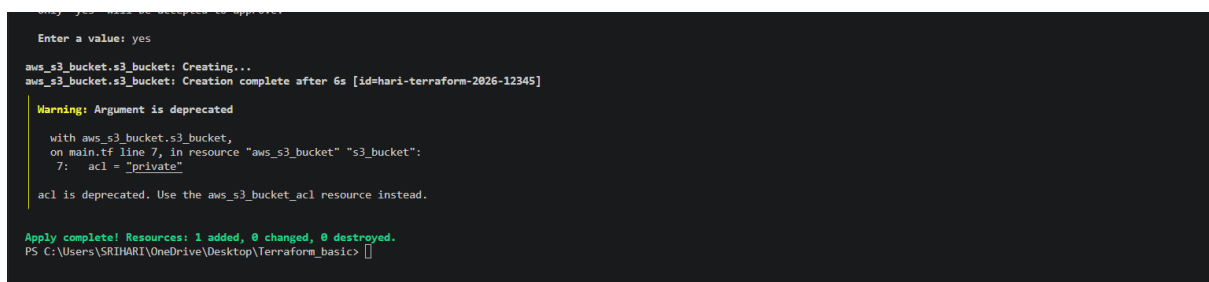


- When Terraform asks:
- Do you want to perform these actions?
- Type:
- yes
- Terraform creates S3 bucket successfully.
- You may see warning:

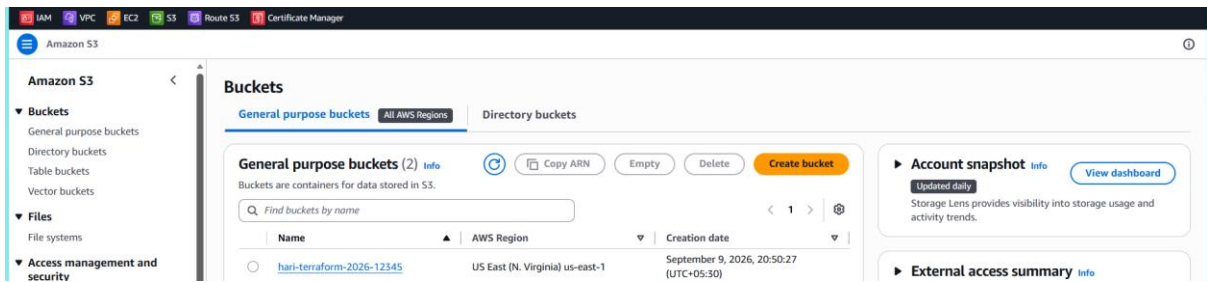
acl is deprecated

- Confirm success line:

Apply complete! Resources: 1 added



- Login to **AWS Console**.
- Go to:
- **S3 → Buckets**
- Search bucket name: hari-terraform-2026-12345
- Confirm bucket exists in the bucket list



- Open **main.tf** in VS Code.
- Keep existing S3 bucket code.
- Add DynamoDB table resource below it:

```
resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {

  name      = "terraform-state-lock-dynamo"

  hash_key   = "LockID"

  read_capacity = 20

  write_capacity = 20

  attribute {

    name = "LockID"

    type = "S"

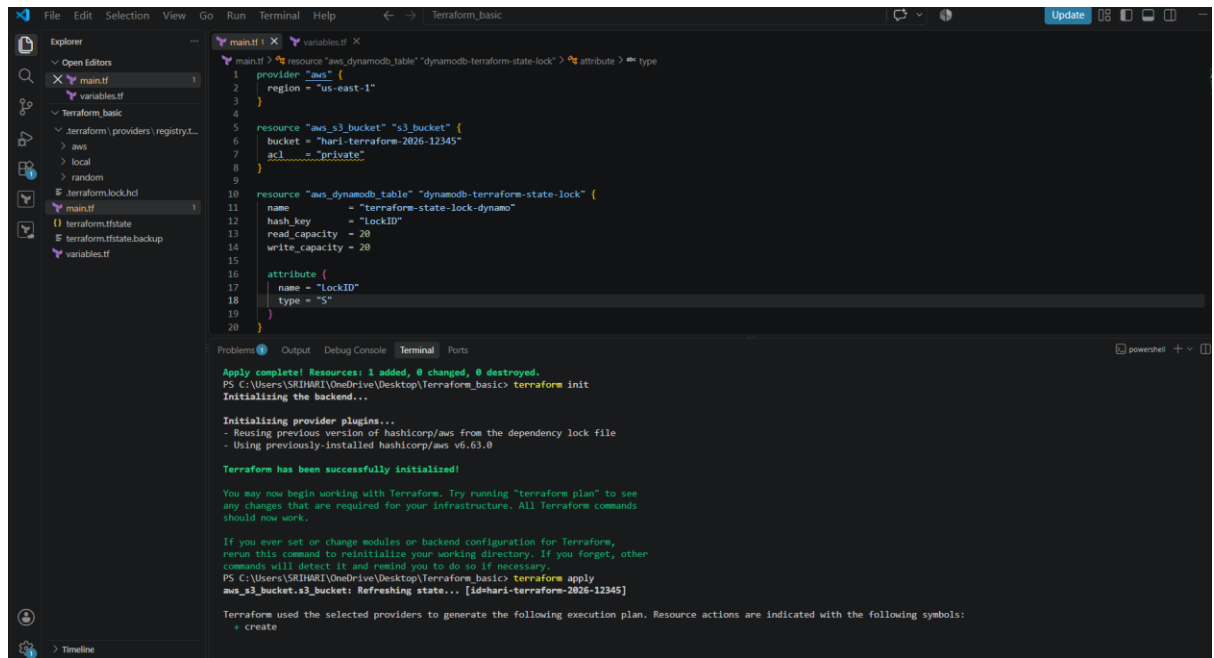
  }

}
```

- Save the file.



- Open terminal in VS Code.
- Run again:
- terraform init
- Confirm initialization is completed successfully.



```
main.tf:1: resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" > attribute > type
1 provider "aws" {
2   region = "us-east-1"
3 }
4
5 resource "aws_s3_bucket" "s3_bucket" {
6   bucket = "hari-terraform-2026-12345"
7   acl    = "private"
8 }
9
10 resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
11   name      = "terraform-state-lock-dynamo"
12   hash_key  = "LockID"
13   read_capacity = 20
14   write_capacity = 20
15
16   attribute {
17     name = "LockID"
18     type = "S"
19   }
20 }
```

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform init

Initializing the backend...

Initializing provider plugins...

- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.63.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

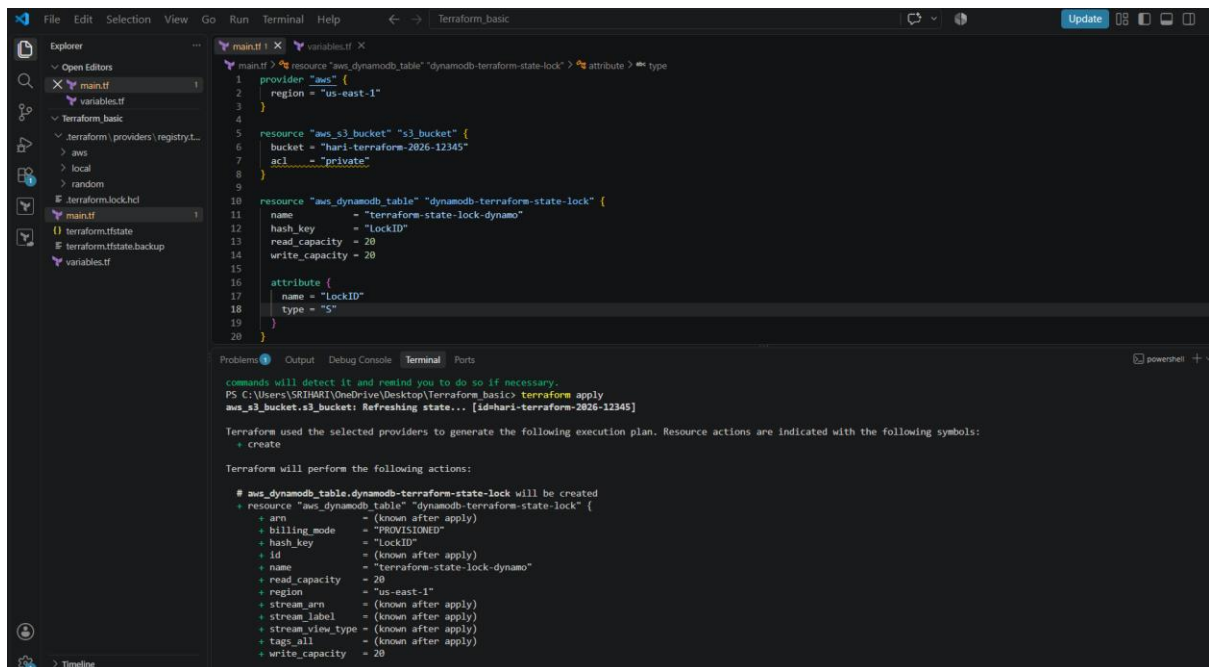
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform apply

aws_s3_bucket.s3_bucket: Refreshing state... [id=hari-terraform-2026-12345]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

- Run:
- terraform apply
- Terraform shows plan and indicates:
- aws_dynamodb_table.dynamodb-terraform-state-lock will be created
- Proceed to approval step



```
main.tf:1: resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" > attribute > type
1 provider "aws" {
2   region = "us-east-1"
3 }
4
5 resource "aws_s3_bucket" "s3_bucket" {
6   bucket = "hari-terraform-2026-12345"
7   acl    = "private"
8 }
9
10 resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
11   name      = "terraform-state-lock-dynamo"
12   hash_key  = "LockID"
13   read_capacity = 20
14   write_capacity = 20
15
16   attribute {
17     name = "LockID"
18     type = "S"
19   }
20 }
```

PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform apply

aws_s3_bucket.s3_bucket: Refreshing state... [id=hari-terraform-2026-12345]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# aws_dynamodb_table.dynamodb-terraform-state-lock will be created
+ resource "aws_dynamodb_table" "dynamodb-terraform-state-lock" {
+   arn                = (known after apply)
+   billing_mode       = "PROVISIONED"
+   hash_key           = "LockID"
+   id                 = (known after apply)
+   name               = "terraform-state-lock-dynamo"
+   read_capacity       = 20
+   region             = "us-east-1"
+   stream_arn         = (known after apply)
+   stream_label       = (known after apply)
+   stream_view_type    = (known after apply)
+   tags_all           = (known after apply)
+   write_capacity      = 20
}
```

```
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

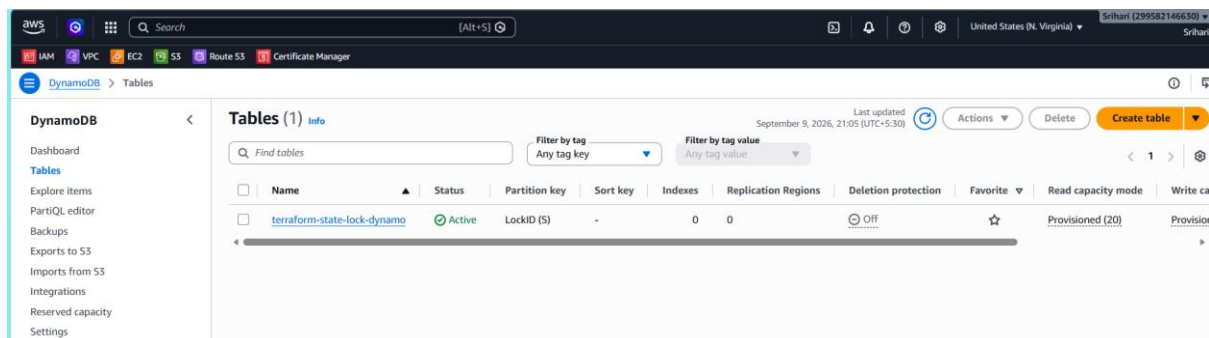
Enter a value: yes

aws_dynamodb_table.dynamodb-terraform-state-lock: Creating...
aws_dynamodb_table.dynamodb-terraform-state-lock: Creation complete after 9s [id:terraform-state-lock-dynamo]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\SRIDHARI\OneDrive\Desktop\Terraform_basics>
```

- Login to **AWS Console**.
- Go to:
DynamoDB → Tables
- Confirm table exists:

terraform-state-lock-dynamo
- Status should show **Active**



- Open **main.tf**.
- Replace deprecated S3 ACL line with proper ACL resource:

```
resource "aws_s3_bucket" "s3_bucket" {  
  
    bucket = "hari-terraform-2026-12345"  
  
}
```

```
resource "aws_s3_bucket_acl" "bucket_acl" {  
  
    bucket = aws_s3_bucket.s3_bucket.id  
  
    acl = "private"  
  
}
```

Keep DynamoDB resource for state locking.

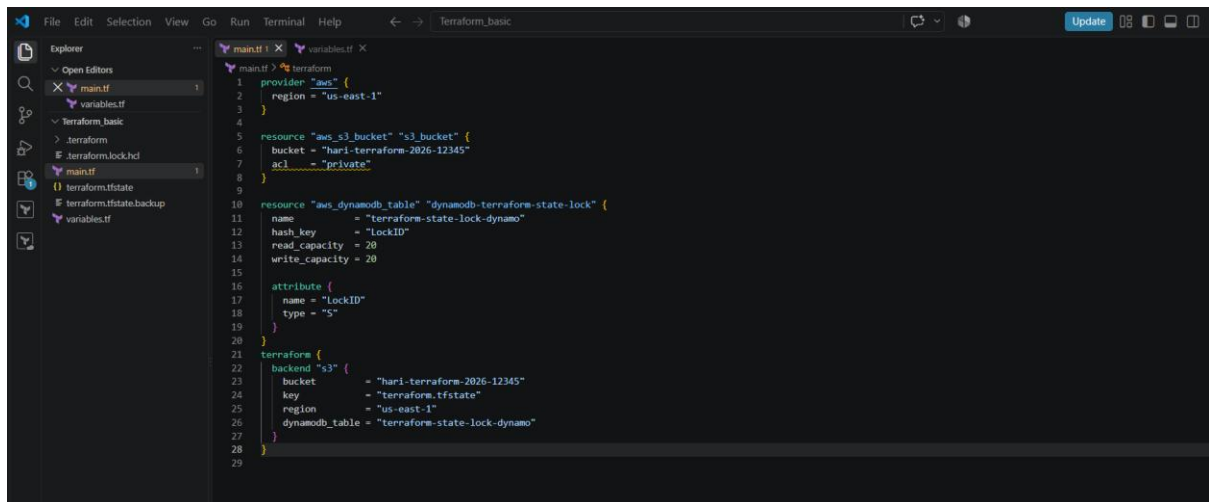
Add backend block for storing Terraform state in S3:

```
terraform {  
  
    backend "s3" {  
  
        bucket      = "hari-terraform-2026-12345"  
  
        key         = "terraform.tfstate"  
  
        region      = "eu-east-1"  
  
        dynamodb_table = "terraform-state-lock-dynamo"
```

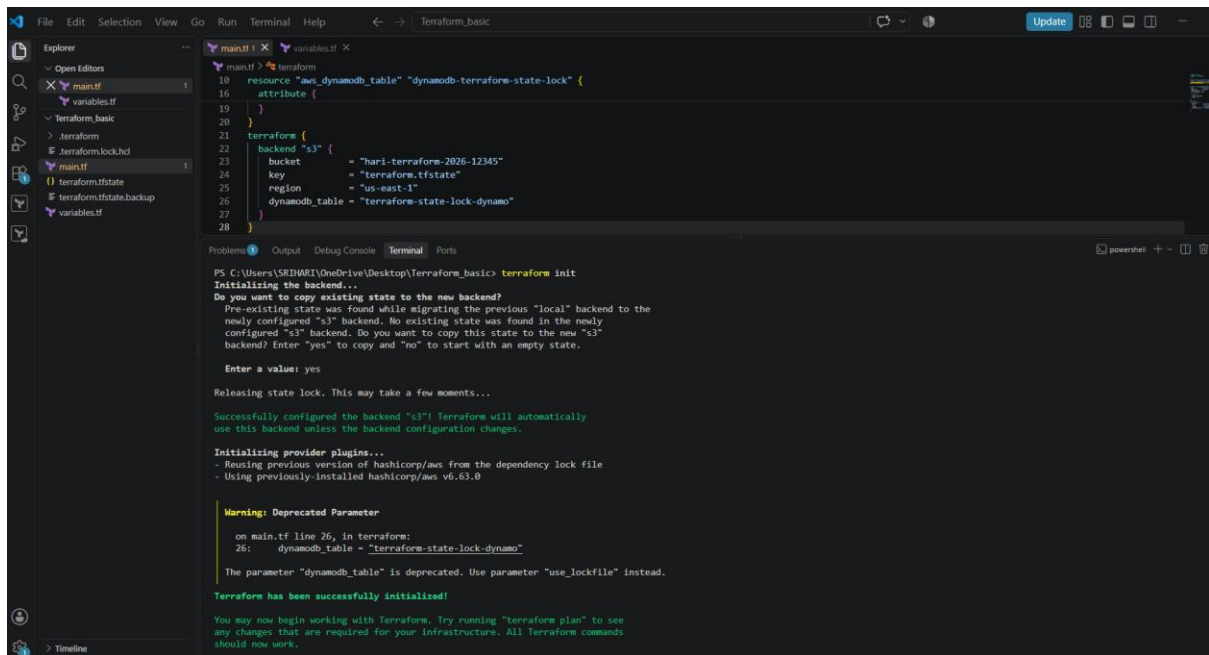
```
}  
  
}
```

- Save file.
- Run:
- terraform init

Terraform migrates state to S3 and enables locking using DynamoDB.

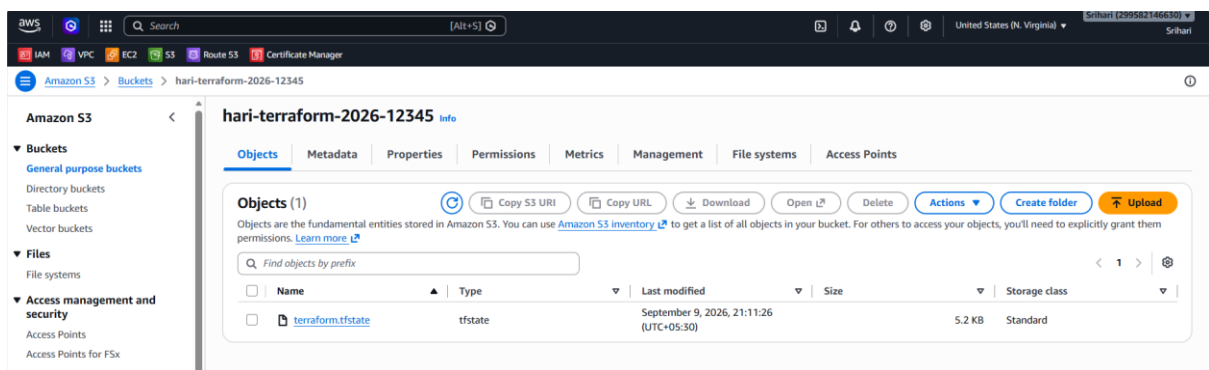


- Open **VS Code** → **Terminal** inside your Terraform project.
- Run:
- terraform init
- Terraform detects backend change and asks:
"Do you want to copy existing state to the new backend?"
- Type:
 - yes
- Terraform completes backend setup and shows:
- **Successfully configured the backend "s3"**
- **Terraform has been successfully initialized**



- Go to **AWS Console**
- Open:
S3 → Buckets → hari-terraform-2026-12345
- Click **Objects**
- Confirm the state file exists:

terraform.tfstate
- This means Terraform state is now stored remotely in S3.



- Open **main.tf** in VS Code.
- Add EC2 resource code:

provider "aws" {

 region = "us-east-1"

}

resource "aws_instance" "webserver" {

```
ami      = "ami-0e3a96b0b22670c57"
```

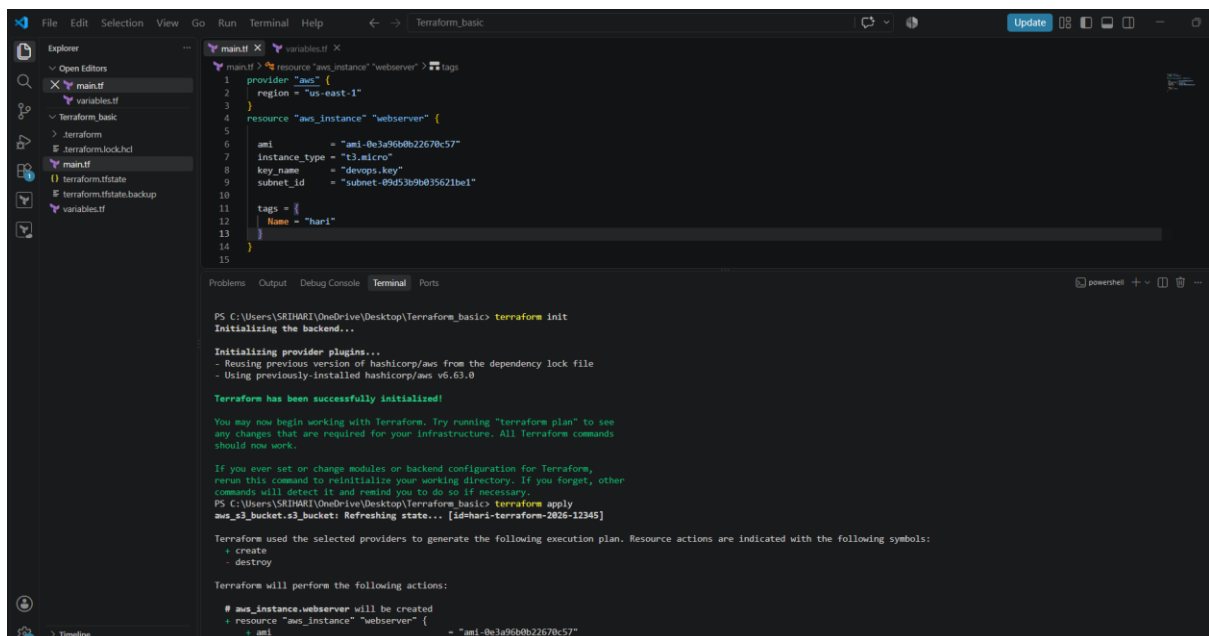
```
instance_type = "t3.micro"
```

```
key_name  = "devops.key"
```

```
subnet_id = "subnet-09d53b9b035621be1"
```

```
tags = {  
  Name = "hari"  
}
```

- Save the file.
- Run:
- terraform apply
- Terraform shows the plan output for EC2 creation.



The screenshot shows the Visual Studio Code interface with a Terraform configuration file open. The configuration defines an AWS instance named 'webserver' with specific AMI, instance type, key name, subnet, and tags. The terminal window shows the execution of 'terraform init' and 'terraform apply' commands. The output of 'terraform apply' shows the plan for creating the instance and its associated resources.

```
File Edit Selection View Go Run Terminal Help  
main.tf X variables.tf X  
1 provider "aws" {  
2   region = "us-east-1"  
3 }  
4 resource "aws_instance" "webserver" {  
5  
6   ami      = "ami-0e3a96b0b22670c57"  
7   instance_type = "t3.micro"  
8   key_name  = "devops.key"  
9   subnet_id = "subnet-09d53b9b035621be1"  
10  
11   tags = {  
12     Name = "hari"  
13   }  
14 }  
15  
Problems Output Debug Console Terminal Ports  
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform init  
Initializing the backend...  
  
Initializing provider plugins...  
- Reusing previous version of hashicorp/aws from the dependency lock file  
- Using previously-installed hashicorp/aws v6.63.0  
  
Terraform has been successfully initialized!  
  
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.  
  
If you ever set or change modules or backend configuration for Terraform,  
rerun this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.  
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic> terraform apply  
aws_s3_bucket.s3_bucket: Refreshing state... [id=hari-terraform-2026-12345]  
  
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:  
+ create  
- destroy  
  
Terraform will perform the following actions:  
  
# aws_instance.webserver will be created  
+ resource "aws_instance" "webserver" {  
  ami      = "ami-0e3a96b0b22670c57"
```

- When asked:
- Do you want to perform these actions?
- Type:
- yes
- Wait for creation to finish.
- Confirm output:
- **Creation complete**

- **Apply complete! Resources: 1 added**

```

1 provider "aws" {
2   region = "us-east-1"
3 }
4 resource "aws_instance" "webserver" {
5   ami           = "ami-0e3a968b22670c57"
6   instance_type = "t3.micro"
7 }
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

```

- id = "i-03a32dec996cd28f4"
- permissions = [
  - "FULL_CONTROL",
] -> null
- type = "CanonicalUser" -> null
# (1 unchanged attribute hidden)

- server_side_encryption_configuration {
  - rule {
    - bucket_key_enabled = false -> null
    - apply_server_side_encryption_by_default {
      sse_algorithm = "AES256" -> null
      # (1 unchanged attribute hidden)
    }
  }
}

- versioning {
  - enabled = false -> null
  - mfa_delete = false -> null
}

Plans: 1 to add, 0 to change, 1 to destroy.

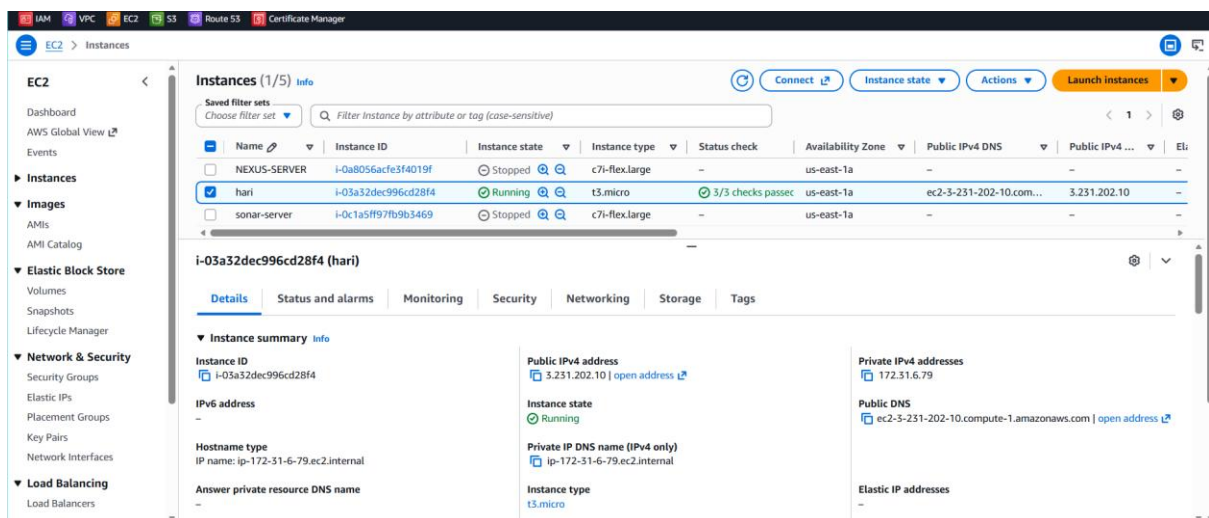
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.s3_bucket: Destroying... [id=hari-terraform-2026-12345]
aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [0m10s elapsed]
aws_instance.webserver: Creation complete after 19s [id=i-03a32dec996cd28f4]

```

- Go to **AWS Console**
- Open: **EC2 → Instances**
- Confirm the instance is visible and status is:
- **Running**
- You can verify details like:
- AMI ID
- Instance Type: t3.micro
- Public IP / DNS



- In VS Code terminal, run:
- terraform destroy
- Terraform shows destroy plan:
aws_instance.webserver will be destroyed
- Continue to approval step.

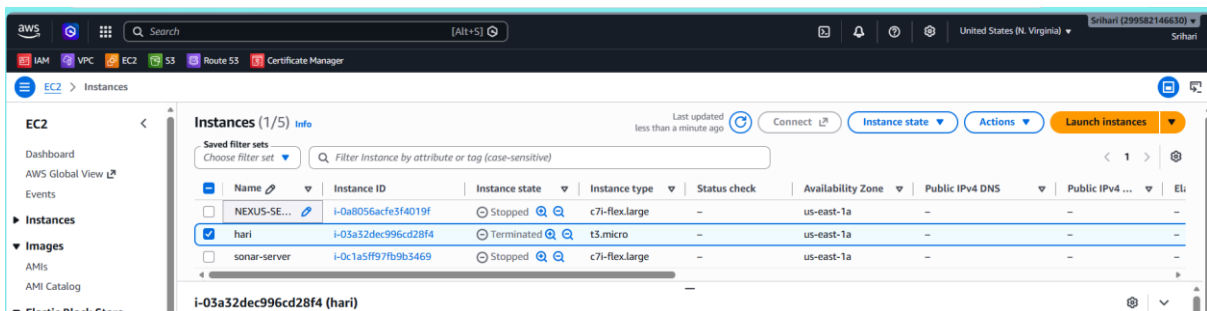
The screenshot shows the VS Code interface with a terminal window open. The terminal displays the output of the command `terraform destroy`. It shows the refresh state for the `aws_instance.webserver` resource and then the execution plan. The plan indicates that the resource will be destroyed. The details of the resource are listed below:

```

# aws_instance.webserver will be destroyed
- resource "aws_instance" "webserver" {
  ami              = "ami-0e3a96b0622670c57" -> null
  instance_type    = "t3.micro"               -> null
  key_name         = "harry"                  -> null
  subnet_id        = "subnet-15288663"        -> null
  vpc_security_group_ids = ["sg-013c096f3f49eb51e"] -> null
  availability_zone = "us-east-1a"            -> null
  associate_public_ip_address = true          -> null
  ebs_optimized    = false                    -> null
  force_destroy    = false                    -> null
  get_password_data = false                    -> null
  hibernation      = false                     -> null
  id               = "i-03a32dec996cd28f4"    -> null
  instance_initiated_shutdown_behavior = "stop" -> null
  instance_state   = "running"                 -> null
  instance_type    = "t3.micro"                 -> null
  ipv6_address_count = 0                       -> null
  ipv6_addresses   = []                       -> null
  key_name         = "harry"                    -> null
  monitoring       = false                     -> null
  placement_partition_number = 0               -> null
  primary_network_interface_id = "eni-013c096f3f49eb51e" -> null
  private_dns      = "ip-172-31-6-79.ec2.internal" -> null
  private_ip       = "172.31.6.79"             -> null
  public_dns       = "ec2-3-231-202-10.compute-1.amazonaws.com" -> null
  public_ip        = "3.231.202.10"            -> null
  region           = "us-east-1"               -> null
  secondary_private_ips = []                   -> null
}

```

- Open AWS EC2 → Instances
- Refresh the page.
- Confirm the instance state changes to:
Shutting-down
then **Terminated**
- This confirms Terraform destroy removed the EC2 resource.



- Open main.tf
- Update EC2 resource to include provisioner:

```

provider "aws" {

  region = "us-east-1"
}

resource "aws_instance" "webserver" {

  ami      = "ami-0e3a96b0b22670c57"

  instance_type = "t3.micro"

  key_name   = "devops.key"

  subnet_id = "subnet-09d53b9b035621be1"

  tags = {

    Name = "hari"

  }

}

provisioner "local-exec" {

  command = "echo Instance ${aws_instance.webserver.public_ip} created! > instance-state"

}

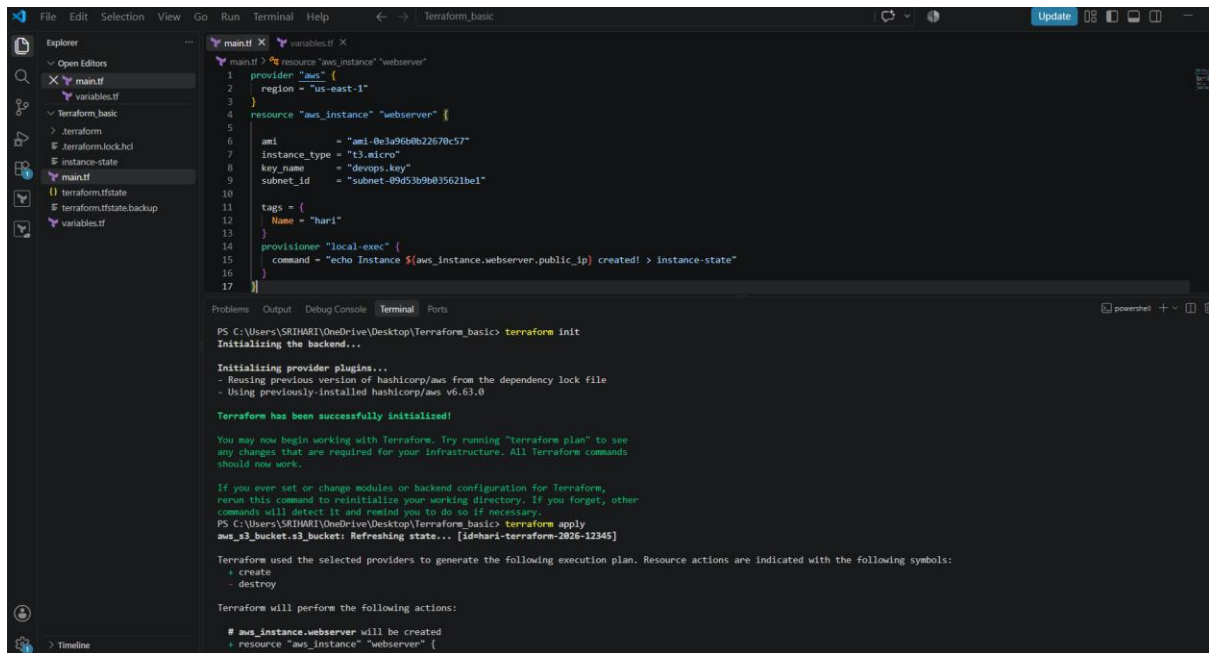
```

- Save the file.
- This will create a local file named **instance-state** after instance creation.



- Run Terraform init (safe even if already initialized):
- terraform init
- Run:
- terraform apply
- Terraform displays plan to create EC2 again.
- Approve when asked by typing:

yes



```
main.tf | variables.tf |  
1 resource "aws_instance" "webserver"  
2 provider "aws" {  
3   region = "us-east-1"  
4 }  
5 resource "aws_instance" "webserver" {  
6   ami           = "ami-0e3a96b0b22670c57"  
7   instance_type = "t3.micro"  
8   key_name      = "devops.key"  
9   subnet_id     = "subnet-09d53b96d35621be1"  
10  
11   tags = {  
12     Name = "hari"  
13   }  
14   provisioner "local-exec" {  
15     command = "echo Instance ${aws_instance.webserver.public_ip} created! > instance-state"  
16   }  
17 }
```

PS C:\Users\SRINIRI\OneDrive\Desktop\Terraform_basic> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.63.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

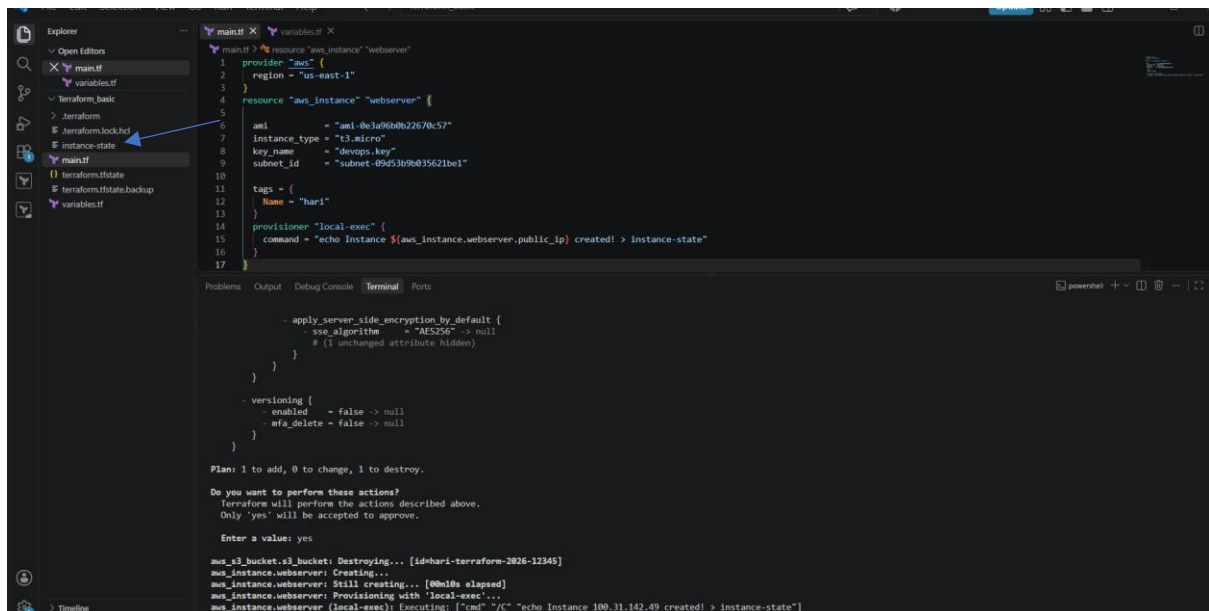
PS C:\Users\SRINIRI\OneDrive\Desktop\Terraform_basic> terraform apply
aws_s3_bucket.s3_bucket: Refreshing state... [id=hari-terraform-2026-12345]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

```
# aws_instance.webserver will be created  
+ resource "aws_instance" "webserver" {
```

- After apply completes, check the **Explorer** panel in VS Code
- You will see a new file created:
- instance-state
- Open it to confirm content is written like:
- Instance <public-ip> created!
- This confirms **local-exec** provisioner worked successfully



```
main.tf | variables.tf |  
1 resource "aws_instance" "webserver"  
2 provider "aws" {  
3   region = "us-east-1"  
4 }  
5 resource "aws_instance" "webserver" {  
6   ami           = "ami-0e3a96b0b22670c57"  
7   instance_type = "t3.micro"  
8   key_name      = "devops.key"  
9   subnet_id     = "subnet-09d53b96d35621be1"  
10  
11   tags = {  
12     Name = "hari"  
13   }  
14   provisioner "local-exec" {  
15     command = "echo Instance ${aws_instance.webserver.public_ip} created! > instance-state"  
16   }  
17 }
```

- apply_server_side_encryption_by_default {
 sse_algorithm = "AES256" -> null
 # (1 unchanged attribute hidden)
}

- versioning {
 enabled = false -> null
 mfa_delete = false -> null
}

Plan: 1 to add, 0 to change, 1 to destroy.

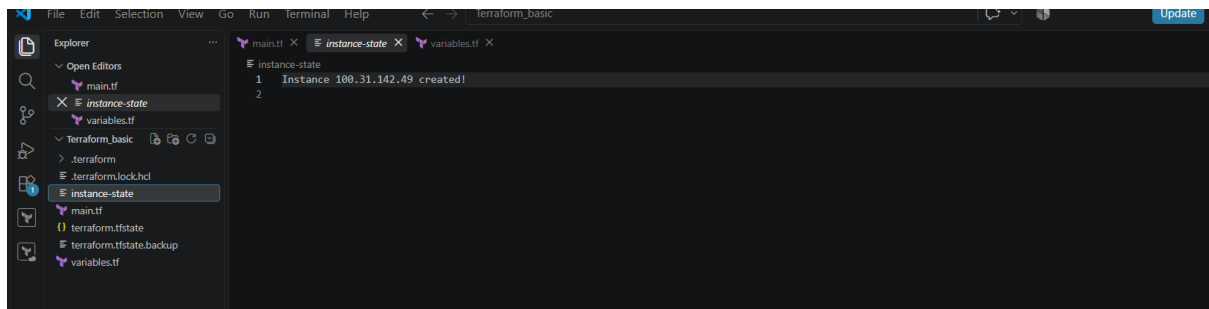
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.s3_bucket: Destroying... [id=hari-terraform-2026-12345]
aws_instance.webserver: Creating...
aws_instance.webserver: Still creating... [0m10s elapsed]
aws_instance.webserver: Provisioning with 'local-exec'...
aws_instance.webserver (local-exec): Executing: [cmd] "C:\> "echo Instance 100.31.142.49 created! > instance-state"

- In VS Code Explorer, click the file:
instance_state
- Open the file and verify the output message is written inside it.

- Example shown in image:
- Instance 100.31.142.49 created!
- This confirms **local-exec** provisioner successfully created and updated the file.



- Open **main.tf**
- Add Local file resource code:

```
resource "local_file" "test" {

  filename = "pets.txt"

  content = "I love cats!"

  provisioner "local-exec" {

    when    = destroy

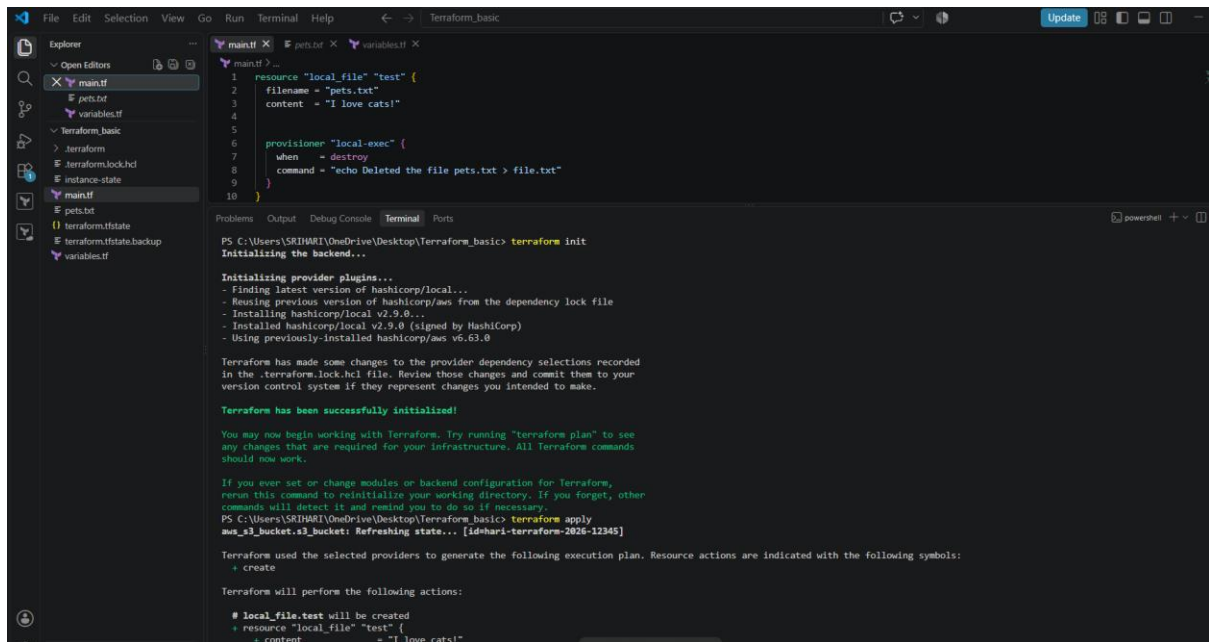
    command = "echo Deleted the file pets.txt > file.txt"

  }

}
```

- Save the file.
- Open terminal and run:
- terraform init
- Then run:
- terraform apply

Terraform shows plan that **local_file.test** will be created.



- In PowerShell terminal set Terraform log file path:

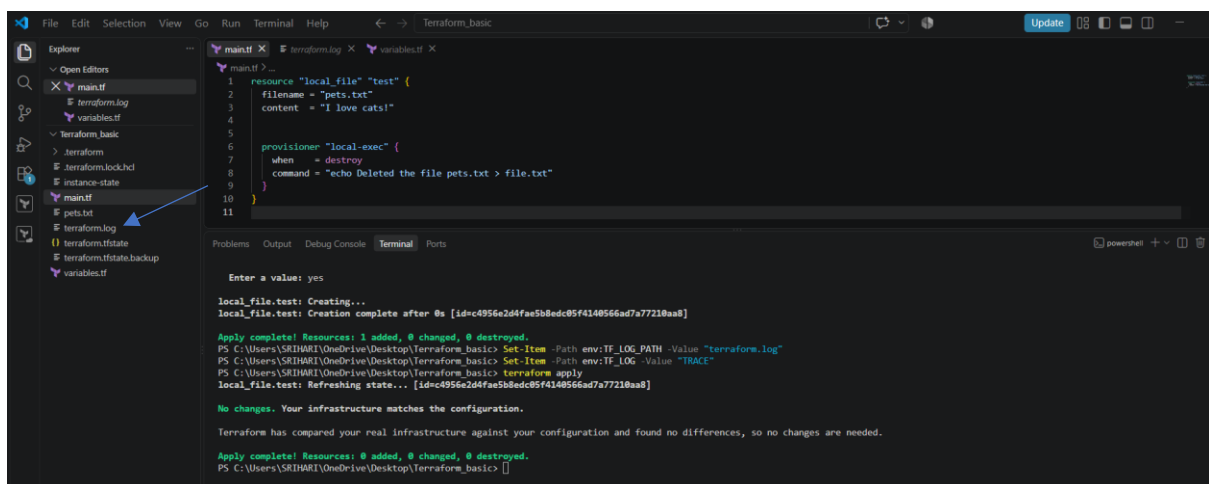
Set-Item -Path env:TF_LOG_PATH -Value "terraform.log"

- Set log level:

Set-Item -Path env:TF_LOG -Value "TRACE"

- Run apply again:
- terraform apply
- Terraform runs with detailed logs enabled.
- A new file appears:

terraform.log



- In VS Code Explorer, click:

terraform.log

- Open the log file.
- Confirm it contains Terraform debugging information like:

Initialization logs

Provider logs

Backend logs

Apply execution logs

- This confirms **TRACE logging is working properly**.

```

1 2026-09-10T15:08:19.737+0530 [INFO] Terraform version: 1.16.0
2 2026-09-10T15:08:19.740+0530 [DEBUG] using github.com/hashicorp/go-tfe v1.108.0
3 2026-09-10T15:08:19.740+0530 [DEBUG] using github.com/hashicorp/hcl/v2 v2.24.0
4 2026-09-10T15:08:19.740+0530 [DEBUG] using github.com/hashicorp/terraform-svchost v0.2.1
5 2026-09-10T15:08:19.740+0530 [DEBUG] using github.com/zclconf/go-cty v1.18.1
6 2026-09-10T15:08:19.740+0530 [INFO] Go runtime version: go1.26.4
7 2026-09-10T15:08:19.740+0530 [INFO] CLI args: [string("C:\Users\SRIHARI\Downloads\terraform\terraform.exe"), "apply"]
8 2026-09-10T15:08:19.740+0530 [TRACE] Stdout is a terminal of width 198
9 2026-09-10T15:08:19.740+0530 [TRACE] Stderr is a terminal of width 198
10 2026-09-10T15:08:19.740+0530 [TRACE] Stdin is a terminal
11 2026-09-10T15:08:19.744+0530 [DEBUG] Attempting to open CLI config file: C:\Users\SRIHARI\AppData\Roaming\terraform.rc
12 2026-09-10T15:08:19.744+0530 [DEBUG] File doesn't exist, but doesn't need to. Ignoring.
13 2026-09-10T15:08:19.744+0530 [DEBUG] ignoring non-existing provider search directory terraform.d/plugins
14 2026-09-10T15:08:19.744+0530 [DEBUG] ignoring non-existing provider search directory C:\Users\SRIHARI\AppData\Roaming\terraform.d/plugins
15 2026-09-10T15:08:19.746+0530 [DEBUG] ignoring non-existing provider search directory C:\Users\SRIHARI\AppData\Roaming\HashiCorp\Terraform\plugins
16 2026-09-10T15:08:19.746+0530 [INFO] CLI command args: [string("apply")]
17 2026-09-10T15:08:19.757+0530 [TRACE] Meta.Backend: no config given or present on disk, so returning nil config
18 2026-09-10T15:08:19.759+0530 [TRACE] Meta.Backend: working directory was previously initialized but has no backend (is using legacy remote state?)
19 2026-09-10T15:08:19.759+0530 [TRACE] Meta.Backend: using default local state only (no backend configuration, and no existing initialized backend)
20 2026-09-10T15:08:19.759+0530 [TRACE] Meta.Backend: instantiated backend of type <nil>
21 2026-09-10T15:08:19.760+0530 [TRACE] providercache.fillMetaCache: scanning directory .terraform/providers
22 2026-09-10T15:08:19.764+0530 [TRACE] getproviders.SearchLocalDirectory: found registry.terraform.io/hashicorp/aws v6.63.0 for windows_amd64 at .terraform/providers/registry.ter
23 2026-09-10T15:08:19.767+0530 [TRACE] getproviders.SearchLocalDirectory: found registry.terraform.io/hashicorp/local v2.2.3 for windows_amd64 at .terraform/providers/registry.te
24 2026-09-10T15:08:19.768+0530 [TRACE] getproviders.SearchLocalDirectory: found registry.terraform.io/hashicorp/local v2.3.0 for windows_amd64 at .terraform/providers/registry.te
25 2026-09-10T15:08:19.768+0530 [TRACE] getproviders.SearchLocalDirectory: found registry.terraform.io/hashicorp/local v2.9.0 for windows_amd64 at .terraform/providers/registry.te
26 2026-09-10T15:08:19.770+0530 [TRACE] getproviders.SearchLocalDirectory: found registry.terraform.io/hashicorp/random v3.9.0 for windows_amd64 at .terraform/providers/registry.te
27 2026-09-10T15:08:19.770+0530 [TRACE] providercache.fillMetaCache: including .terraform/providers/registry.terraform.io/hashicorp/aws v6.63.0 for windows_amd64 as a candidate package
28 2026-09-10T15:08:19.770+0530 [TRACE] providercache.fillMetaCache: including .terraform/providers/registry.terraform.io/hashicorp/local v2.2.3 for windows_amd64 as a candidate packag
29 2026-09-10T15:08:19.770+0530 [TRACE] providercache.fillMetaCache: including .terraform/providers/registry.terraform.io/hashicorp/local v2.3.0 for windows_amd64 as a candidate packag
30 2026-09-10T15:08:19.770+0530 [TRACE] providercache.fillMetaCache: including .terraform/providers/registry.terraform.io/hashicorp/local v2.9.0 for windows_amd64 as a candidate packag
31 2026-09-10T15:08:19.770+0530 [TRACE] providercache.fillMetaCache: including .terraform/providers/registry.terraform.io/hashicorp/random v3.9.0 for windows_amd64 as a candidate packag
32 2026-09-10T15:08:21.988+0530 [TRACE] providercache.fillMetaCache: using cached result from previous scan of .terraform/providers
33 2026-09-10T15:08:22.023+0530 [DEBUG] checking for provisioner in "."
34 2026-09-10T15:08:22.023+0530 [DEBUG] checking for provisioner in "C:\Users\SRIHARI\Downloads\terraform"
35 2026-09-10T15:08:22.824+0530 [TRACE] Meta.Backend: backend (nil) does not support operations, so wrapping it in a local backend
36 2026-09-10T15:08:22.831+0530 [INFO] backend/local: starting Apply operation
37 2026-09-10T15:08:22.831+0530 [TRACE] backend/local: requesting state manager for workspace "default"
38 2026-09-10T15:08:22.831+0530 [TRACE] backend/local: state manager for workspace "default" will:
39   - read initial snapshot from terraform.tfstate
40   - write new snapshots to terraform.tfstate
41

```

3. Create one **EC2 instance** with **httpd** installed using a Terraform script.

- Open **main.tf**
- Add/update EC2 instance code with user data script:

```
provider "aws" {
```

```
  region = "us-east-1"
```

```
}
```

```
resource "aws_instance" "webserver" {
```

```
ami      = "ami-0e3a96b0b22670c57"
```

```
instance_type = "t3.micro"
```

```
key_name  = "devops.key"
```

```
subnet_id = "subnet-09d53b9b035621be1"
```

```
user_data = <<-EOF
```

```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y httpd
```

```
systemctl start httpd
```

```
systemctl enable httpd
```

```
echo "<h1>Terraform EC2 Web Server</h1>" > /var/www/html/index.html
```

```
EOF
```

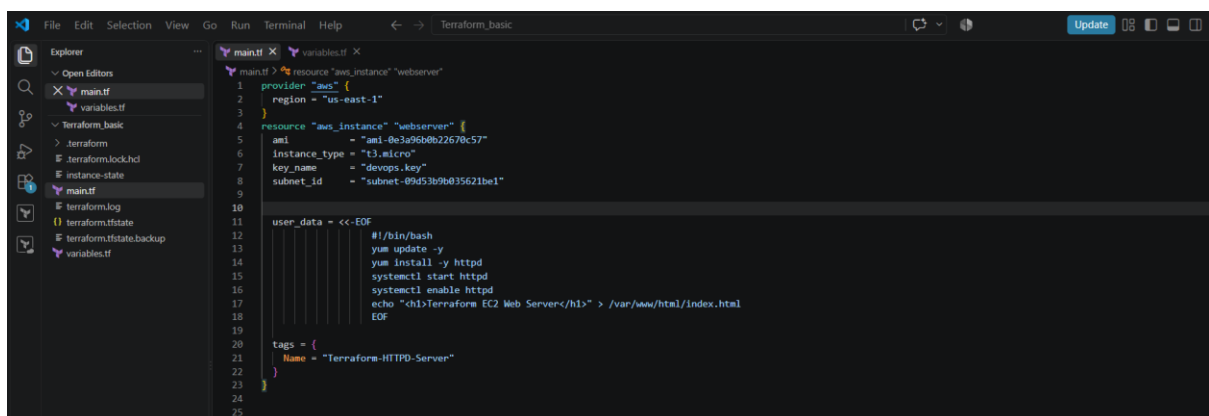
```
tags = {
```

```
    Name = "Terraform-HTTPD-Server"
```

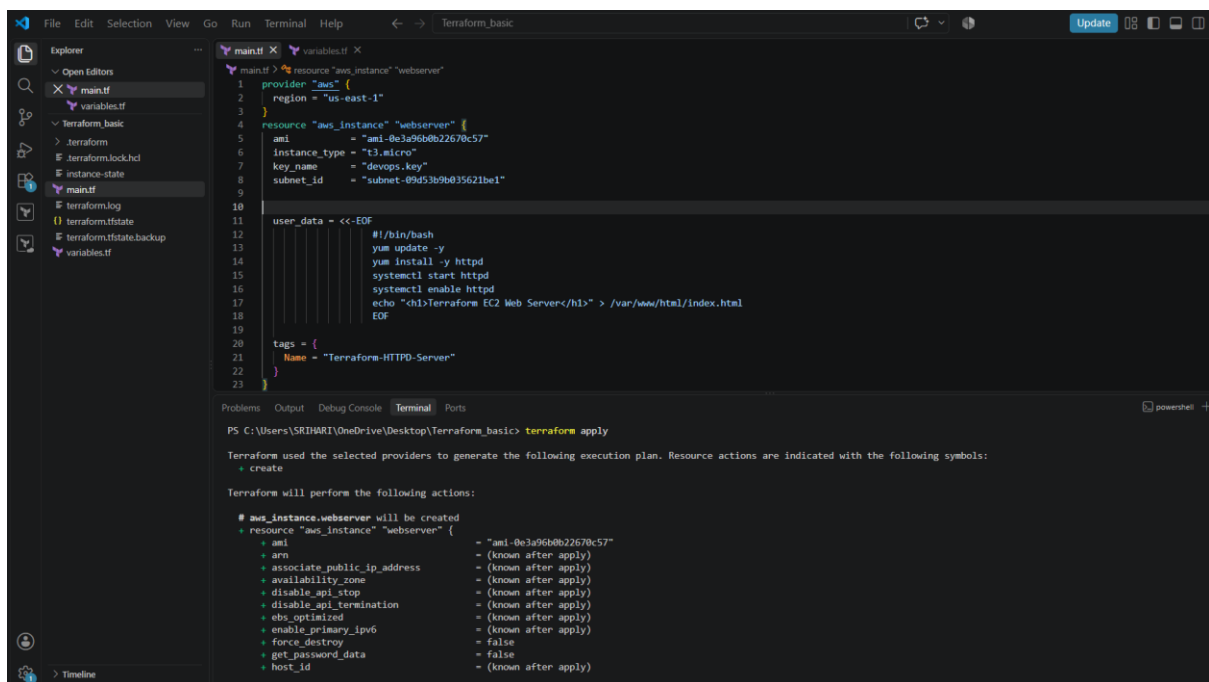
```
}
```

```
}
```

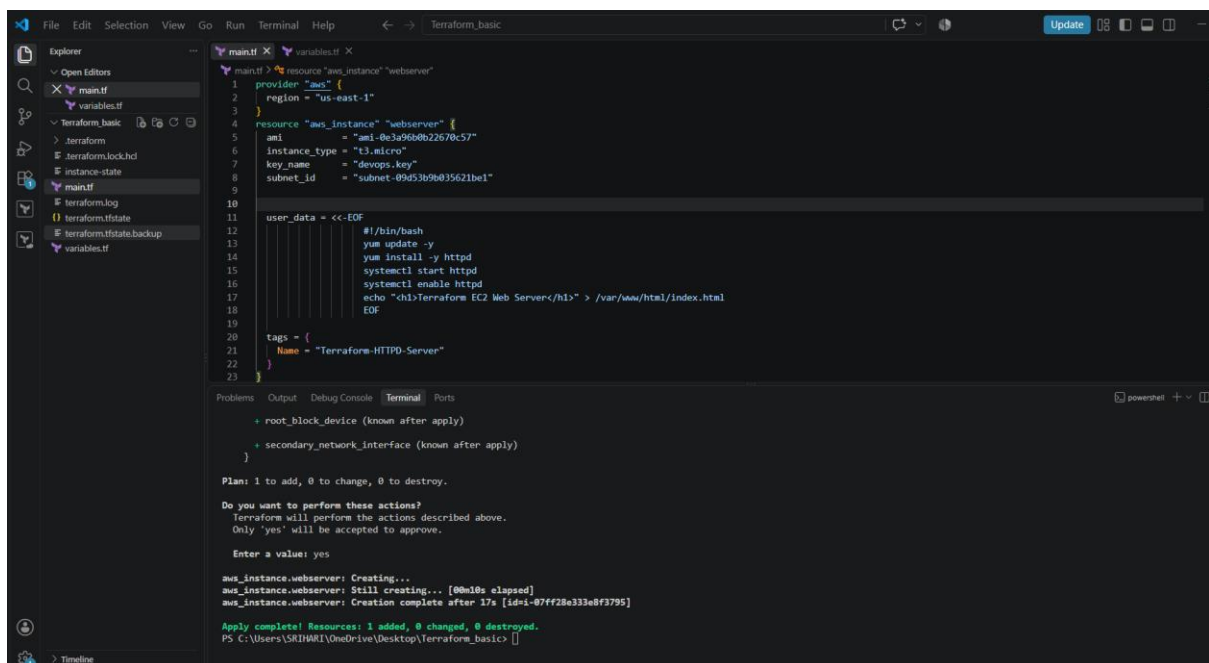
- Save the file.



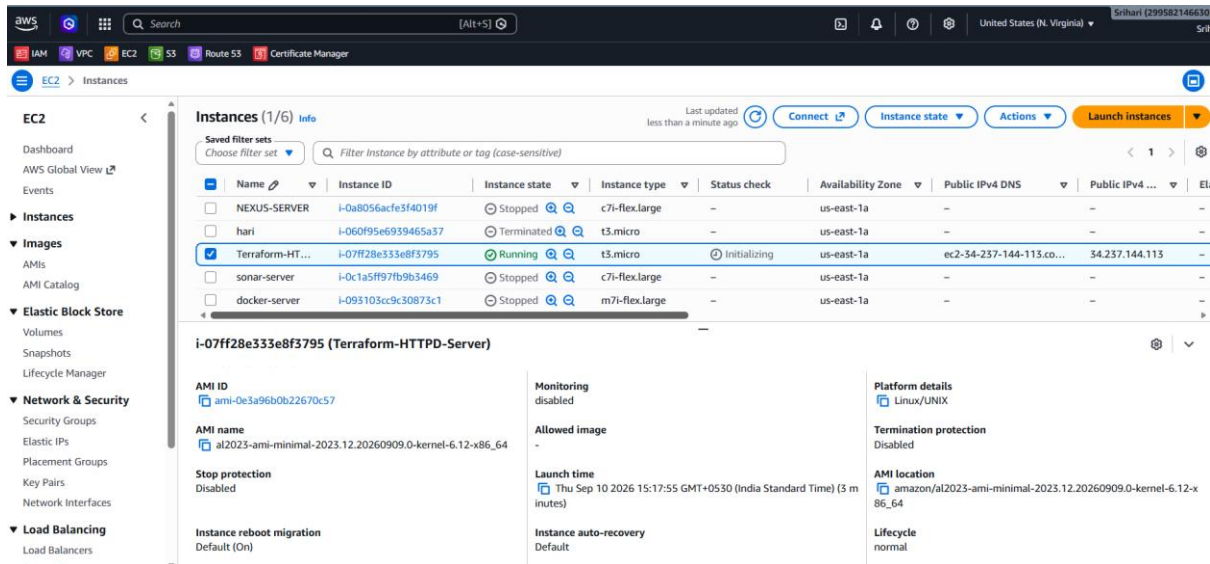
- Open terminal in VS Code.
- Run:
 - terraform apply
 - Terraform shows it will create:
- aws_instance.webserver
- Continue to approval step



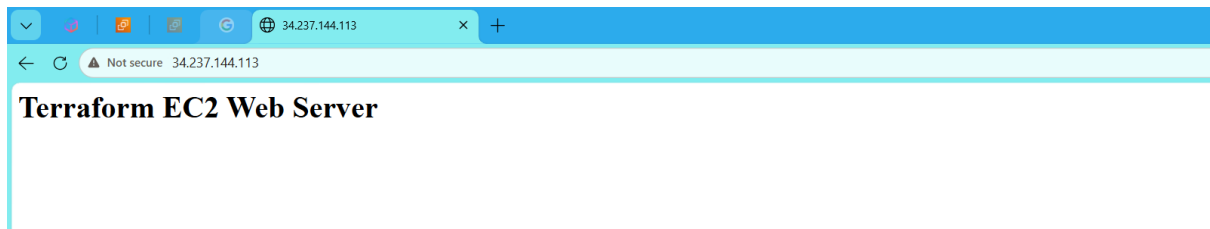
- Terraform asks:
- Do you want to perform these actions?
- Type:
- yes
- Wait until Terraform finishes creating the instance.
- Confirm success message:
- **Apply complete! Resources: 1 added**



- Open **AWS Console**
- Go to:
EC2 → Instances
- Confirm the new instance is running:
- Name: Terraform-HTTPD-Server
- Status: **Running**
- Check the instance **Public IPv4 address** (shown in the table).

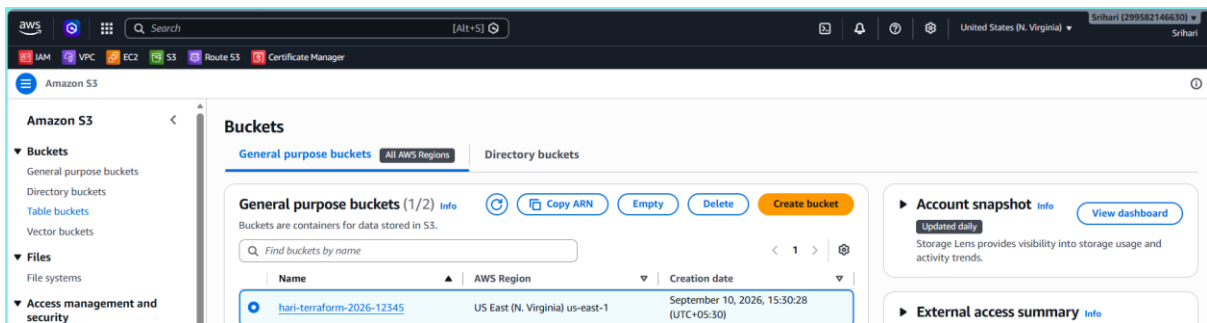


- Copy the EC2 **Public IPv4 address**
- Paste it in browser:
- http://<PUBLIC-IP>
- Page should load and display:
- Terraform EC2 Web Server
- This confirms:
- Apache (httpd) installed
- Web server running
- index.html created through user_data



4. Set up S3 as backend for task 3.

- Login to **AWS Console**
- Go to **S3 → Buckets**
- Confirm the bucket is created:
- mujahed-terraform-backend-2026
- Verify the bucket region is:
- **United states (N.Virgina) us-east-1**
- This bucket will be used to store Terraform remote state

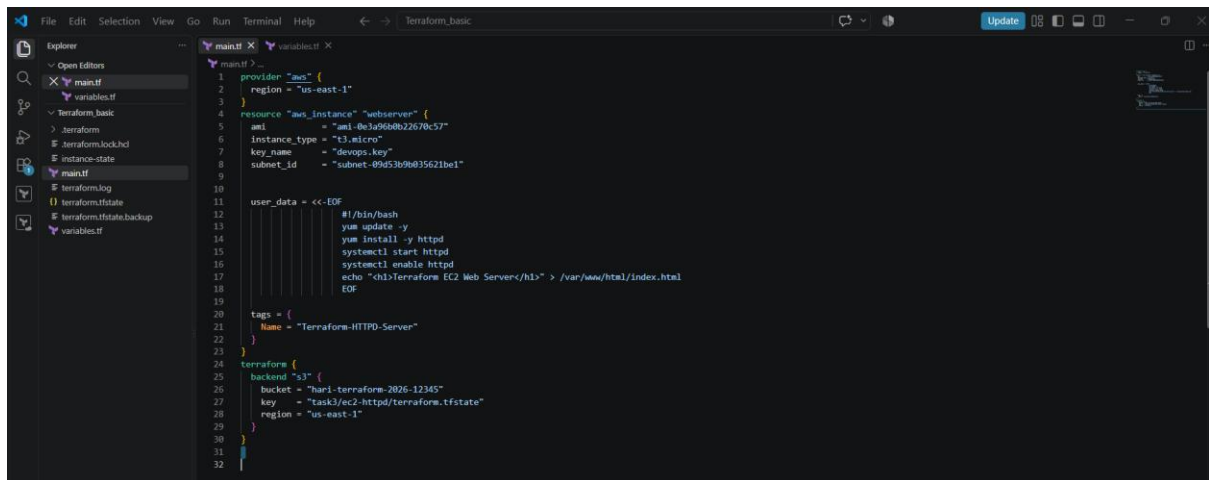


- Open **main.tf** in VS Code.
- Scroll to the bottom and add backend block:

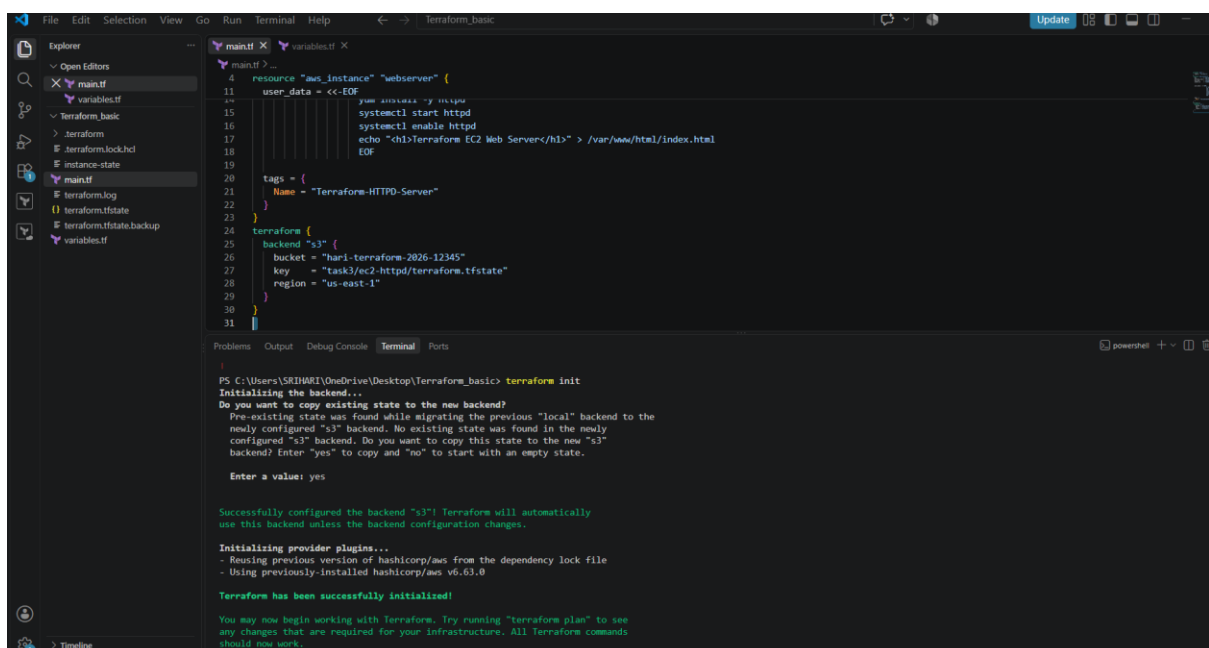
```
terraform {  
  
  backend "s3" {  
  
    bucket = " hari-terraform-2026-12345"  
  
    key   = "task3/ec2-httpd/terraform.tfstate"  
  
    region = "es-east-1"  
  
  }  
  
}
```

- Save the file.

This ensures Terraform state is stored remotely in S3.



- Open VS Code Terminal.
- Run:
 - terraform init
 - Terraform asks:
 - Do you want to copy existing state to the new backend?
 - Type:
 - yes
 - Confirm message appears:
 - Successfully configured the backend "s3"
 - Terraform has been successfully initialized



- In terminal, run:
 - terraform apply
 - Terraform checks AWS resources using remote state.

- You see output like:

No changes. Your infrastructure matches the configuration.

- This means everything is already created and matching.

```

4 resource "aws_instance" "webservers" {
11   user_data = <<EOF
12   |
13   | yum install -y httpd
14   | systemctl start httpd
15   | systemctl enable httpd
16   | echo "ch!Terraform EC2 Web Server</h1>" > /var/www/html/index.html
17   | EOF
18
19   tags = {
20     Name = "Terraform-HTTPD-Server"
21   }
22
23   terraform {
24     backend "s3" {
25       bucket = "hari-terraform-2026-12345"
26       key     = "task3/ec2-httpd/terraform.tfstate"
27       region = "us-east-1"
28     }
29   }
30 }
31

```

Problems Output Debug Console Terminal Ports

```

- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.63.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform apply
aws_instance.webservers: Refreshing state... [id=i-07ff28e333e8f3795]

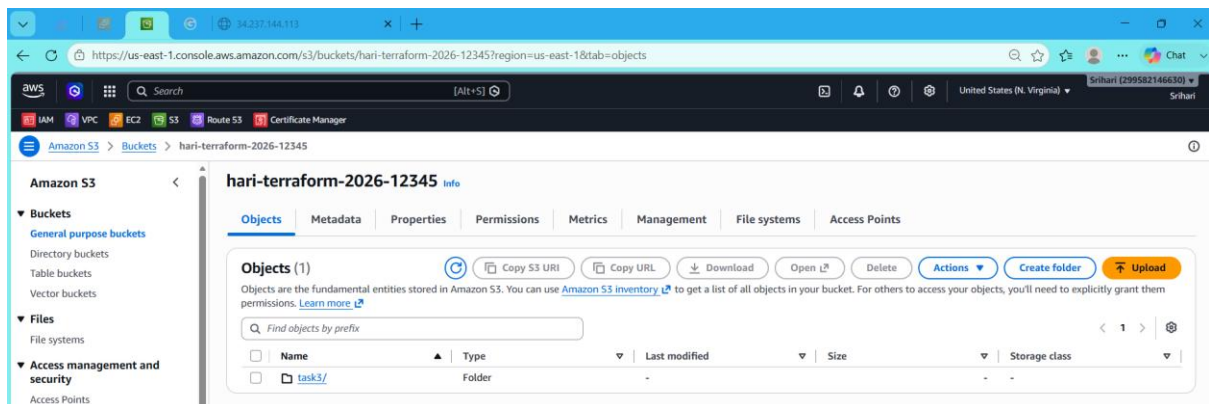
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> ]

```

- Terraform completes the execution.
- Confirm final message:
- **Apply complete! Resources: 0 added, 0 changed, 0 destroyed.**
- This confirms:
- Backend is working properly
- Terraform state is stable in S3
- No updates required



QWS

Search

[Alt+S]

United States (N. Virginia)

Srihari (299582146630)

IAM

VPC

EC2

S3

Route 53

Certificate Manager

Amazon S3

Buckets

hari-terraform-2026-12345

task3/

ec2-httpd/

terraform.tfstate

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

terraform.tfstate

Info

Copy S3 URI

Download

Open

Object actions

Properties

Permissions

Versions

Object overview

Owner

189b273016da414d49198516a7b868cd7b9fa80de91a50a61ed605edc5e4bcd5

AWS Region

US East (N. Virginia) us-east-1

Last modified

September 10, 2026, 15:45:00 (UTC+05:30)

Size

5.5 KB

Type

tfstate

Key

task3/ec2-httpd/terraform.tfstate

S3 URI

s3://hari-terraform-2026-12345/task3/ec2-httpd/terraform.tfstate

Amazon Resource Name (ARN)

arn:aws:s3:::hari-terraform-2026-12345/task3/ec2-httpd/terraform.tfstate

Entity tag (Etag)

b37550ee88ba000ca9d8bd0acfa0b0d8

Object URL

https://hari-terraform-2026-12345.s3.us-east-1.amazonaws.com/task3/ec2-httpd/terraform.tfstate